

For an **Embedded LRU (Line Replaceable Unit) Acceptance Test Plan (ATP)**, **User Flash Testing** verifies that the non-volatile user storage device (NOR Flash / NAND Flash / eMMC / SPI Flash / QSPI Flash / SD/eMMC boot flash depending on architecture) functions correctly for **storage, boot support, reliability, and data integrity**.

Since you asked specifically for **LRU ATP level testing**, I'll structure this in a **production/acceptance qualification format**:

1. Objective
 2. Test Setup
 3. Procedure
 4. Expected Results
 5. Practical execution on real LRU hardware
-

1. USER FLASH ATP — PURPOSE

User Flash ATP validates:

- ✓ Flash device detection
 - ✓ Capacity verification
 - ✓ Read operation
 - ✓ Write operation
 - ✓ Erase operation
 - ✓ Data retention
 - ✓ Address boundary integrity
 - ✓ Bit corruption detection
 - ✓ Throughput/performance
 - ✓ Boot image integrity
 - ✓ Power interruption robustness
 - ✓ ECC/Error handling
 - ✓ Environmental reliability
-

2. TEST SETUP

Typical LRU setup:

Equipment	Purpose
LRU Unit	DUT
Bench Power Supply	Stable input power
JTAG Debugger	Debug access
UART Console	Logging
Host PC	Test scripts
Oscilloscope	Timing verification
Environmental chamber	Temp ATP
Flash programming utility	Program/readback

Example interfaces:

- SPI Flash
- QSPI Flash
- NAND
- NOR
- eMMC
- SD Flash

Processor example:

T2080 SBC

3. USER FLASH ATP TESTS

TEST 1 — FLASH DETECTION TEST

Objective

Verify processor recognizes flash.

Procedure

Power ON LRU.

Run:

```
flash_probe
```

or

U-Boot:

```
sf probe
```

Linux:

```
dmesg | grep flash
```

Processor reads:

- Manufacturer ID
- Device ID
- JEDEC ID

Expected Result

Device detected correctly.

Example:

```
Manufacturer: Micron  
ID:0x20BA18  
Capacity:16MB
```

No timeout.

No CRC error.

Practical LRU execution

Via UART terminal:

```
=> sf probe
```

Observe console.

TEST 2 — FLASH CAPACITY VERIFICATION

Objective

Verify actual flash size.

Procedure

Read device register.

Compare:

Specified size vs detected size.

Example:

Design:

```
Expected=256MB
```

Detected:

```
256MB
```

Commands

Linux:

```
cat /proc/mtd
```

U-Boot:

```
mtd list
```

Expected Result

Capacity matches BOM specification.

No truncation.

TEST 3 — READ TEST

Objective

Verify read operation.

Procedure

Read known data.

Example:

Stored pattern:

```
55AA55AA
```

Execute:

```
flash read 0x10000 256
```

Compare output.

Expected Result

Read data identical.

No bit mismatch.

Real LRU method

Use JTAG memory viewer.

OR

UART:

```
md.b address
```

Compare with golden image.

TEST 4 — WRITE TEST

Objective

Verify programming capability.

Procedure

Create test pattern.

Example:

```
AAAAAAAA  
55555555  
12345678  
FFFFFFFF
```

Write to flash.

Command:

U-Boot:

```
sf write RAM_ADDR FLASH_ADDR SIZE
```

Read back.

Compare.

Expected Result

100% match.

No partial programming.

Real execution

Program temporary sector.

Never overwrite production boot sector.

Use reserved ATP partition.

TEST 5 — ERASE TEST

Objective

Verify erase function.

Procedure

Write known pattern.

Perform erase.

Command:

```
sf erase
```

Read data back.

Expected erased state:

NOR:

0xFF

Sometimes NAND:

0x00 or FF

(dependent on device)

Expected Result

Entire sector erased.

No residual bits.

LRU execution

Choose safe user partition.

Avoid bootloader sectors.

TEST 6 — WRITE-READBCK VERIFY TEST

Objective

Detect hidden programming errors.

Procedure

Write pattern.

Immediately read.

Compare.

Pattern examples:

00000000
FFFFFFFF
AAAAAAAA



55555555
Walking1
Walking0
Pseudo random



Expected Result

Exact byte-level match.

0 mismatches.

TEST 7 — ADDRESS BOUNDARY TEST

Objective

Verify memory decoding.

Procedure

Test:

- First address
- Last address
- Sector boundaries
- Page boundaries

Example:

0x00000000
0x0000FFFF
0x00FFFFFF

Write unique patterns.

Read back.

Expected Result

Correct addressing.

No mirroring.

No aliasing.

TEST 8 — SECTOR TEST

Objective

Verify all sectors functional.

Procedure

Loop through entire flash.

For each sector:

1 Write.

2 Read.

3 Verify.

4 Erase.

Pseudo-code:

```
for(sector=0;sector<MAX;sector++)  
{  
    write();  
    read();  
    compare();  
    erase();  
}
```

Expected Result

100% sector pass.

No bad sector.

TEST 9 — BAD BLOCK TEST (NAND/eMMC)

Objective

Identify defective blocks.

Procedure

Scan flash.

Linux:

```
nand scrub  
nand bad
```

Expected Result

Bad blocks within manufacturer limits.

Example:

```
0-20 factory bad blocks acceptable
```

TEST 10 — DATA RETENTION TEST

Objective

Verify long-term storage reliability.

Procedure

Write pattern.

Power OFF.

Wait:

- 1 hr
- 24 hr
- extended ATP if required

Power ON.

Read back.

Expected Result

Data preserved.

No corruption.

TEST 11 — POWER INTERRUPTION TEST

Objective

Verify flash robustness during sudden power loss.

Procedure

Start write operation.

Cut power mid-write.

Restore power.

Check:

- filesystem integrity
 - partition status
 - boot image integrity
-

Expected Result

Defined recovery behavior.

No uncontrolled corruption.

Real LRU setup

Use programmable PSU.

Trigger OFF during write.

TEST 12 — BOOT IMAGE TEST

Critical for LRU.

Objective

Verify flash boots processor.

Procedure

Store:

- Bootloader
- FPGA image
- firmware image

Cold reboot.

Observe startup.

Expected:

Processor boots successfully.

Monitor:

UART console.

Example:

```
U-Boot start
Kernel loaded
Application running
```

TEST 13 — CRC / CHECKSUM TEST

Objective

Verify image integrity.

Procedure

Compute CRC.

Example:

```
crc32 flash_region
```

Compare with golden CRC.

Expected Result

CRC identical.

TEST 14 — ECC TEST

Important for NAND/eMMC.

Objective

Verify correction logic.

Procedure

Inject single-bit error.

Read back.

Monitor ECC.

Expected:

ECC corrects data.

Reports correction count.

TEST 15 — STRESS / ENDURANCE TEST

Objective

Validate wear reliability.

Procedure

Repeated:

Erase
Write
Read
Verify

Thousands of cycles.

Expected Result

No premature failure.

Meets endurance spec.

TEST 16 — PERFORMANCE TEST

Objective

Measure flash bandwidth.

Procedure

Measure:

Read speed.

Write speed.

Example:

```
dd if=/dev/mtd0 of=/dev/null
```

Expected Result

Within specification.

TEST 17 — TEMPERATURE ATP TEST

Important for aerospace LRU.

Objective

Verify flash under temperature extremes.

Procedure

Run flash ATP at:

```
-40°C  
25°C  
+70°C  
+85°C
```

depending qualification level.

Perform:

- read
 - write
 - erase
 - boot
-

Expected Result

Normal operation.

No timing failure.

TEST 18 — VIBRATION / POWER QUALITY TEST

(Optional ATP / qualification)

Perform flash access under:

- vibration
- voltage variation
- ripple

Expected:

No corruption.

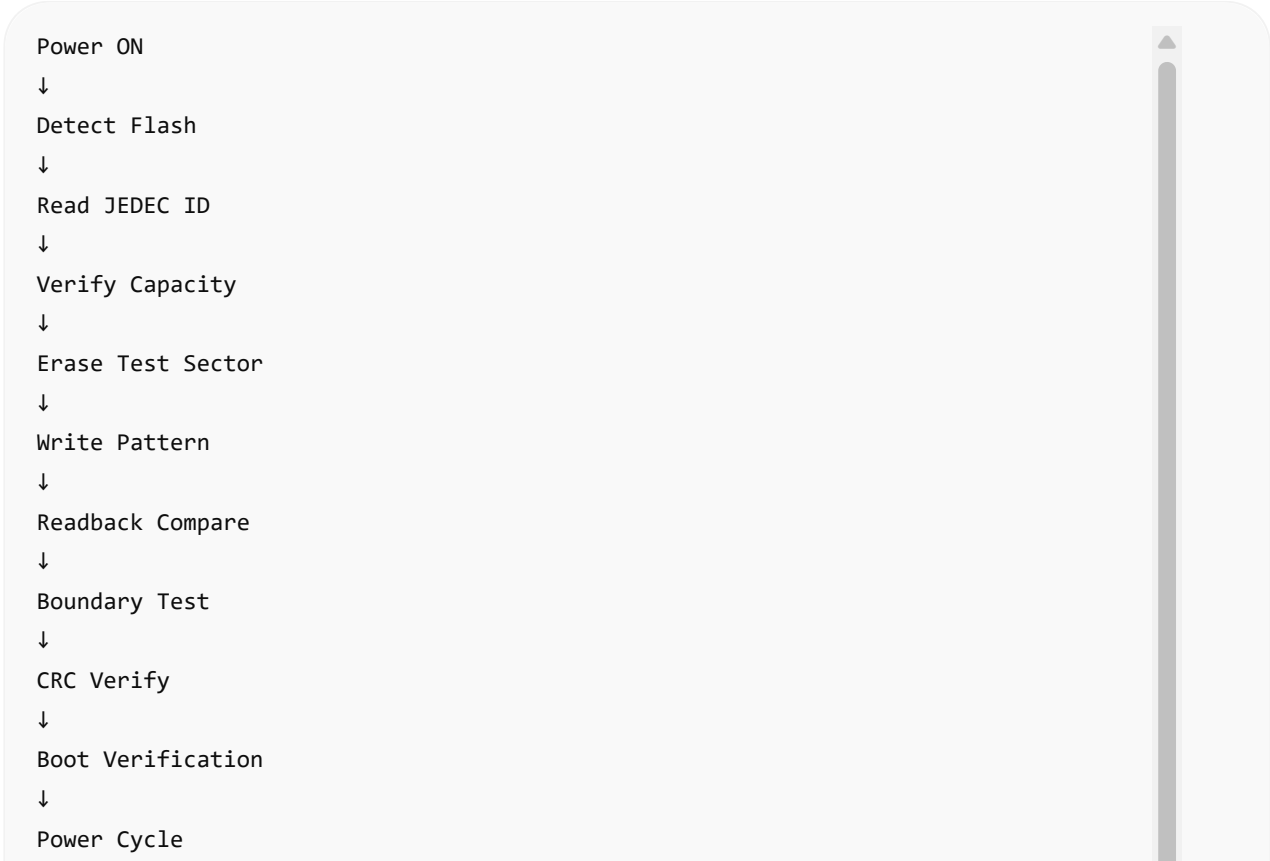
4. COMPLETE USER FLASH ATP MATRIX

Test	Purpose
Detection	Device discovery
Capacity	Size verification
Read	Data access
Write	Programming
Erase	Sector clearing
Verify	Data integrity
Boundary	Address correctness
Sector	Full coverage

Test	Purpose
Bad Block	NAND health
Retention	NVM persistence
Power Loss	Recovery robustness
Boot	Startup capability
CRC	Integrity
ECC	Error correction
Stress	Wear endurance
Performance	Throughput
Temperature	Environmental
Power/Vibration	Qualification

5. HOW THIS IS DONE IN REAL LRU ATP

Typical ATP automation flow:



```
Power ON
↓
Detect Flash
↓
Read JEDEC ID
↓
Verify Capacity
↓
Erase Test Sector
↓
Write Pattern
↓
Readback Compare
↓
Boundary Test
↓
CRC Verify
↓
Boot Verification
↓
Power Cycle
```



Retention Test



PASS/FAIL Report



Usually implemented using:

- Python ATP scripts
- UART automation
- JTAG automation
- LabVIEW/TestStand
- C test firmware

For your T2080 SBC LRU, I can next give a processor-specific User Flash ATP procedure (U-Boot/Linux/JTAG commands for T2080 board) with exact command examples and execution flow.